2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00052

Zebra: Efficient Redundant Array of Zoned Namespace SSDs Enabled by Zone Random Write Area (ZRWA)

Tianyang Jiang[1], Guangyan Zhang[2]*, Xiaojian Liao[3], Yuqi Zhou[4]
[1]*Huawei Technologies Co., Ltd.,* [2]*Department of Computer Science and Technology, BNRist, Tsinghua University,*
[3]*Beihang University,* [4]*China University of Geosciences*
jiangtianyang2@huawei.com, gyzh@tsinghua.edu.cn, liaoxj@buaa.edu.cn, 2104220041@email.cugb.edu.cn

***Abstract*— Zoned Namespace (ZNS) SSDs have emerged as a promising solution for eliminating device-level garbage collection and achieving predictable performance through the zone abstraction, which supports append-only writes. However, integrating ZNS SSDs into RAID configurations presents challenges, particularly concerning partial parity updates (PPUs) due to the no-overwrite property of ZNS SSDs. Existing solutions introduce dedicated metadata zones to manage PPUs, but these zones become performance bottlenecks, especially under high PPU workloads.**

In this paper, we introduce Zebra, a novel architecture for ZNS RAID that leverages the Zone Random Write Area (ZRWA) feature of modern ZNS SSDs. Our key insight is that the SSD's write buffer (or ZRWA) in front of each zone is ideal for PPUs that exhibit a repeated sequential overwrite I/O pattern. By ensuring that the parity chunk fits within the ZRWA window, Zebra enables efficient PPUs within the ZRWA. Combined with a set of techniques, Zebra ensures both high performance and reliability. We evaluate Zebra on both large-zone and small-zone ZNS SSDs, two typical ZNS models. Our experiments with micro and application benchmarks show that Zebra improves the throughput by 1.1×-4.2× compared to RAIZN, a state-of-the-art ZNS RAID system.

I. Introduction

Zoned Namespace (ZNS) SSDs [12], [55] eradicate notorious device-level garbage collection (GC) and achieve predictable performance by providing the zone abstraction, which allows append-only writes instead of random in-place updates [1]. From a software perspective, many applications (*e.g.*, RocksDB [15], F2FS [40]) favor append writes on SSDs, making them well-suited for adaptation to ZNS with minimal patches and promoting the rapid deployment and development of ZNS community [63].

The benefits of ZNS SSDs come with challenges, especially when organizing them into RAID for increased throughput and reliability. The concept of ZNS RAID, as introduced in RAIZN [37], involves a logical volume manager that presents a single logical ZNS SSD [2] to applications, while striping data and parity chunks across multiple physical ZNS SSDs (Figure 1(a)). During a full stripe write, data chunks of a logical zone are distributed to individual physical zones and the parity chunk is calculated and written once. However, partial stripe writes, which are smaller than the stripe size and common (accounting for 75% of the block-level write requests) in data centers [32], [47], [48], [79], introduce a Partial Parity Update (PPU) [28], [75] and incur performance penalties [19]. More seriously, PPUs are particularly problematic to ZNS due to the no-overwrite property of the ZNS SSD, complicating parity management. For example (Figure 1(a)), when appending a new data chunk to ZNS_2, the ZNS RAID system needs to read the old parity chunk from ZNS_1 (①), recalculate the parity (②), and then update the parity in place (③). However, as ZNS cannot overwrite the parity chunk, managing parity in ZNS RAID becomes non-trivial.

To solve the problem and make ZNS RAID realistic, RAIZN creates dedicated *metadata zones* managing the parity chunk (Figure 1(b)). The key idea of RAIZN is to buffer PPUs in a metadata zone. However, our experiments reveal that metadata zones in RAIZN become the performance bottleneck and decrease the throughput by up to 26% (§II). The root cause is that the metadata zone, similar to the file system journaling, suffers from the contention of multi-zone PPU aggregation, the RAID-level GC used to reclaim obsolete PPUs, and the write amplification caused by PPU log headers. We evaluate RAIZN under various workloads with different PPU ratios (Figure 2) and find that as the PPU ratio in the workload increases, RAIZN throughput decreases by up to 59%. What's even worse is that for some ZNS models with hardware isolation [10], [56] (*i.e.*, each zone occupies one dedicated flash channel), metadata zones reduce the maximum available bandwidth to applications by up to 76%.

We present Zebra, a new architecture for ZNS RAID. Our key contribution is a novel use of Zone Random Write Area (ZRWA) [13], [59], an existing ZNS feature, for RAID PPUs, which overwrites parity chunks as in traditional RAID and eliminates the need for metadata zones. With a close look into SSDs, we find that the append-only interface of ZNS is overly strict and can not expose the raw characteristics of the SSD controller. Specifically, modern SSDs employ RAM-based write buffers in front of flash pages or blocks [49]. Data

*Guangyan Zhang is the corresponding author.

[1]In-place updating of SSD refers to overwriting data on the level of the block device instead of flash pages in this paper.

[2]This paper focuses on ZNS RAID that presents a standard ZNS interface as its append-only characteristic is popular in modern advanced storage systems.

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00052

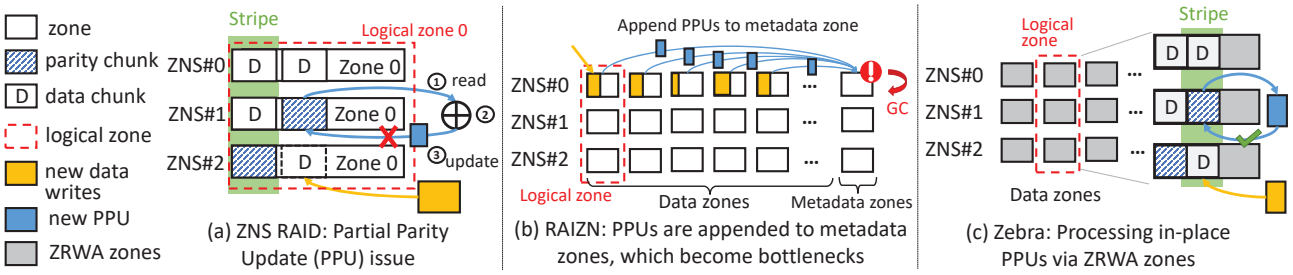


Fig. 1. Three architectures of ZNS RAID. (a) Vanilla ZNS RAID appends partial stripe writes in data chunks but cannot perform in-place updates in parity chunks due to ZNS append-only semantics, leading to the issue of PPU. (b) RAIZN appends PPUs to per-ZNS metadata zones, which makes metadata zones the performance bottleneck and introduces the RAID-level GC. (c) Zebra processes in-place PPUs in parity chunks via ZRWA and eliminates metadata zones.

blocks smaller than the size of the flash page or block are first stored in the write buffer. When the buffer becomes full, data blocks are sent to the flash page or blocks in a batch. Hence, **the write buffer in front of the flash media is a natural fit for PPUs that feature repeated overwrites in ZNS RAID**.

Fortunately, ZRWA exposes the write buffer in front of a zone to the host, supporting in-place updates within a section of contiguous address space (i.e., ZRWA window). Zebra leverages this ZRWA feature in modern ZNS SSDs by setting the chunk size not to exceed the size of the ZRWA window. This approach allows PPUs to be performed repeatedly within the ZRWA window. Once a stripe is filled, Zebra commits the parity chunk to the underlying flash media and advances to the next ZRWA window, thereby maintaining the append-only semantic of ZNS. From the hardware perspective, Zebra separates data and parity on the write buffer and prevents parity from being prematurely committed to flash blocks through the ZRWA abstraction exposed by ZNS.

Realizing a full-fledged ZNS RAID atop ZRWA is a non-trivial task. Zebra introduces a set of techniques including crash-consistent write pointer maintenance and fast machine-failure recovery to ensure high performance and reliability.

We evaluate Zebra on two typical ZNS models (*i.e.*, both large-zone and small-zone ZNS SSDs) to show the performance improvement. Our experiments show that, compared to prior work RAIZN [37], Zebra improves the throughput by 6.4%-51% on large-zone ZNS RAID and achieves a significant throughput increase of 4.2× on small-zone ZNS SSD RAID. Meanwhile, Zebra cuts the average latency by 17%-24%. For application benchmarks, Zebra increases the throughput of RocksDB by 5.4× at most. In summary, we make the following contributions in this paper:

- We demonstrate that the Zone Random Write Area (ZRWA) is well-suited for write requests with a repeatedly sequential overwrite pattern. Furthermore, our research identifies Partial Parity Updates (PPUs) in ZNS RAID as a killer application for ZRWA.
- We conduct an in-depth study of the state-of-the-art RAID system, RAIZN, and uncover a significant performance bottleneck caused by the metadata zones.
- We introduce Zebra, a ZNS RAID system that leverages ZRWA to store PPUs, effectively addressing a range of issues associated with ZRWA.
- The performance improvement brought by Zebra is verified on both large-zone and small-zone ZNS SSDs via both micro and application benchmarks.

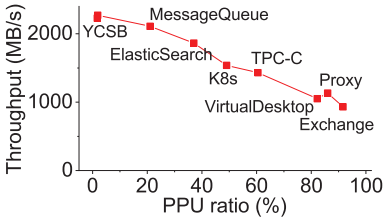


Fig. 2. The impact of Partial Parity Updates on the throughput of ZNS RAID systems. Workloads have different PPU ratios.

II. Background and Motivation

A. The ZNS Interface

Traditional block-interface SSDs suffer from unpredictable performance due to uncontrollable device-side GC behavior [12], consuming the internal bandwidth of SSDs. To alleviate the device-level GC and make it controllable, ZNS SSDs are proposed to shift the data placement policy and the GC to the host software. Meanwhile, the ZNS software community is getting increasingly active, with many file systems [24], [42], [68] and key-value stores [33], [43] running directly on ZNS SSDs. Many mainstream cloud vendors also have deployed ZNS SSDs in their storage systems at scale [54], [80].

Zone abstraction. ZNS provides zone abstraction [58]: like a log structure, data blocks are only allowed to append to the tail of each zone, which is maintained by a pointer called *write pointer* (WP) and marks the end of the last write. The data in a zone before WP are immutable (*i.e.*, read-only). A new write request can only be appended after the WP, and upon successful completion, the WP is incremented by the size of the request. ZNS provides linear address space to upper software, similar to traditional block devices. The difference is that a write request to ZNS fails if its logical block address (LBA) does not match WP or exceeds the zone size.

Each zone has an associated status describing the current state of the zone [58]. A zone starts in the *empty* state, transitions into the *open* state when data are written into the

zone, and finally becomes the *full* state when the last writable block in the zone is written. Zones can be reset to the *empty* state, which erases all data in zones and rewinds WPs to the beginning of zones. A zone not in *empty* and *full* states are collectively referred to *active* states. ZNS SSDs have limited active zones (e.g., 14 in Western Digital ZN540) due to the limited SSD resources (e.g., write buffer).

B. ZNS RAID

ZNS RAID consists of multiple physical ZNS SSDs and presents them as a single logical SSD. This logical SSD can function as either a traditional block device, allowing random overwrites, or as a ZNS SSD, which restricts operations to append-only writes. This paper focuses on the later append-only interface as it simplifies the development and management and is favored by modern advanced storage systems such as Alibaba Pangu [50] and Facebook Tectonic [60].

The address space of logical zones is organized in the form of *stripes* (Figure 1(a)). Each stripe comprises `k+m` fixed-size contiguous areas called *chunks* that reside in `k+m` ZNS SSDs (one chunk per drive), consisting of `k` data chunks and `m` parity chunks. We take RAID 5 as an example in this paper (*i.e.*, `m`=1). When writing to data chunks, parity chunks in the same stripe should be modified synchronously to ensure the data consistency of stripes so that data can be restored after a disk failure. ZNS RAID only allows read requests and append-only write requests issued by applications. Random writes to ZNS RAID are not allowed. When the size of write requests is smaller than the size of stripes, PPU occurs.

Figure 1(a) shows the examples of PPU in ZNS RAID. A write request to ZNS RAID comprises three steps: 1) reading the old parity 2) calculating the new parity 3) writing the new data and parity. Writing the new parity requires in-place updates, which are supported by block-interface SSDs but not by ZNS SSDs.

An ideal ZNS RAID system should achieve high throughput, low latency and fast recovery at the same time. RAIZN [37] and ZapRAID [46], [66] are two recent works on ZNS RAID. ZapRAID sidesteps the PPU issue of ZNS RAID by buffering small-sized write requests into full-stripe writes in DRAM and processing them in a batch. The batching approach increases the write request latency of applications because ZapRAID does not return the aggregated requests to applications unless all of them are issued to devices (see details in III-D). Besides, ZapRAID cannot achieve fast recovery because it requires full-disk retrieval after a power-off event.

RAIZN is another popular ZNS RAID system. Given the close alignment of RAIZN with our work, we provide a detailed examination of it here. RAIZN has several types of metadata including parity log entries and RAID configuration parameters. To avoid in-place PPUs, RAIZN allocates one physical *metadata zone* per ZNS SSD to store parity log entries generated by all logical zones (Figure 1(b)). Similar to the traditional log structure, every persisted metadata log entry in RAIZN contains a log header. When the metadata zone becomes full, RAIZN performs RAID-level GC to reclaim free

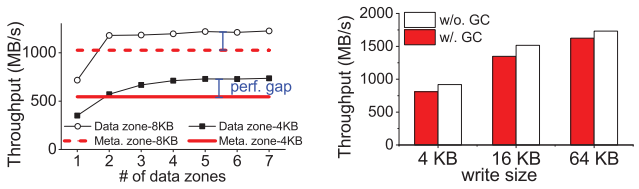


Fig. 3. Performance issues of RAIZN. (a) The 4 KB / 8 KB write throughput gap between one metadata zone and multiple data zones. (b) Garbage collection (GC) impact on RAIZN throughput under 4 KB / 16 KB / 64 KB write workloads. The throughput of RAIZN drops when GC of the metadata zone kicks in.

space. The metadata zone introduces three main performance issues, leading to low throughput and slow recovery.

1) Contention of Multi-Zone PPU Aggregation: The contention of multi-zone PPU aggregation occurs when multiple logical zones in RAIZN receive write requests and generate PPUs for the single metadata zone (Figure 1(b)).

However, in one ZNS device, the sole metadata zone does not offer enough bandwidth for multiple data zones and becomes the performance bottleneck. We test the performance of a popular ZNS SSD, Western Digital (WD) ZN540 [69]. Under a 4 KB sequential write workload, we observe that the throughput of the ZNS SSD with 6 data zones peaks at around 730 MB/s (Figure 3(a)). By contrast, the peak throughput of a single metadata zone is only 545 MB/s (similar results found in [46]). It results in a performance gap of around 185 MB/s between metadata and data zones while processing PPUs. Similar results can be found under 8 KB sequential write workload. The performance gap finally incurs a 19% and 16% throughput reduction under 4 KB and 8 KB write workload, respectively.

Moreover, the contention of metadata zones becomes more serious in other ZNS models like Samsung PM1731a [61]. It is because in such ZNS SSDs, flash blocks of a zone are only redirected to one channel and the bandwidth of a metadata zone is strictly limited to the bandwidth of the channel that the metadata zone is located on (more details in IV-A). It further increases the bandwidth gap between metadata zones and data zones. Our experiments show that the throughput of RAIZN is bounded by the bandwidth of metadata zones, leading to a 76% throughput decrease (Figure 12).

Adding more metadata zones for PPUs can not remove the performance bottleneck and instead introduce extra side effects. ZNS SSDs have limited active zones, so adding more metadata zones reduces the number of data zones, thereby lowering the maximum available bandwidth that applications can utilize. Even with optimizations, the throughput of an enhanced RAIZN version with more metadata zones remains below the ideal (Figure 18(b)). Furthermore, fewer data zones for applications potentially cause issues with optimization strategies like multi-log and multi-zone space allocation, which recommend using more data zones to increase the application throughput [51], [77]. Last, adding metadata zones

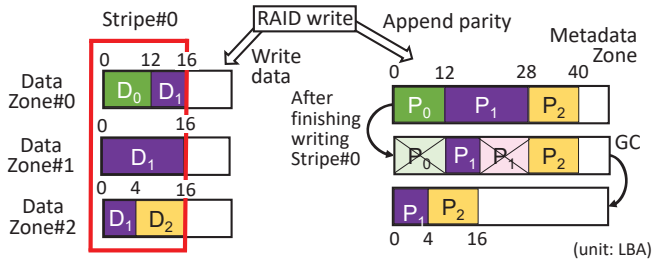


Fig. 4. GC process of RAIZN metadata zones. The crossed parity parts are obsolete when the stripe becomes full and their space is reclaimed.

incurs the extra space overhead.

Implications. The current 'all-to-one' design, where all data zones write PPUs to one metadata zone, causes contention and is suboptimal for ZNS RAID. This finding motivates the exploration of an 'all-to-all' design, where all data zones can write PPUs in parallel, as a potential improvement.

2) Garbage Collection: In traditional RAID systems, the space ratio of data and parity is `k:m` (recalling that a stripe consists of `k` data chunks and `m` parity chunks). Considering the append-only semantics of ZNS, RAIZN utilizes out-of-place updates to store PPUs in metadata zones, which incurs the extra space overhead. To maintain the same space proportion of parity as traditional RAID, RAIZN has to perform GC periodically to clean obsolete PPUs.

Figure 4 shows how obsolete PPUs are created. Here stripes are configured with 3+1 (3 data chunks plus 1 parity chunk), and the chunk size is 16 LBAs. Data $Zone_0$, $Zone_1$, $Zone_2$ form the *Logical* $Zone_0$. The first request D_0 with the size of 12 LBAs comes. RAIZN appends D_0 to $Zone_0$ and calculates the partial parity P_0, the same size as D_0, followed by appending P_0 to the metadata zone. The second request is D_1 with the size of 24 LBAs, so RAIZN splits D_1 into 3 sub-I/Os and issues them to $Zone_0$, $Zone_1$, $Zone_2$, respectively. Meanwhile RAIZN appends the PPU of D_1 (*i.e.*, P_1) to the metadata zone. The size of P_1 is 16 LBAs because the updated parity size never exceeds the parity chunk size itself. Finally, the third request D_2 comes with the size of 12 LBAs, finishing the data write of $Stripe_0$. Also, the corresponding PPU, P_2 (12 LBAs) is appended. To mitigate the space occupancy of parity (16 LBAs theoretically but 40 LBAs now), RAIZN traverses PPUs in the metadata zone, finding that only the first 4 LBAs of P_1 and the entire P_2 are valid. So RAIZN marks other PPUs as obsolete and reclaims them, consuming extra bandwidth of SSDs.

We evaluate the impact of GC on RAIZN's performance (Figure 3(b)). The *w/.-GC* setup triggers GC during the test while the *w/o.-GC* represents the performance when GC is not triggered. By comparing *w/.-GC* and *w/o.-GC*, we find that the GC in RAIZN reduces the throughput by up to 15% under 16 KB sequential write workload. Moreover, GC increases the 99 percentile (P99) latency from 26µs to 43µs in RAIZN.

Implications. GC hurts the performance efficiency and stability of ZNS RAID, motivating us to update parity in place and eliminate GC when designing a ZNS RAID.

TABLE I
ZRWA SETTINGS OF COMMERCIAL ZNS SSDS.

ZNS SSD models	Zone size	ZRWA size	ZRWAFG
Western Digital ZN540 [69]	1077 MB	1 MB	16 KB
DapuStor J5100 [22]	8928 MB	8 MB	32 KB
Samsung PM1731a [61]	96 MB	64 KB	32 KB

3) Write Amplification of Log Headers: If ZNS RAID utilizes journaling to buffer PPUs, log headers are necessary since the information such as parity range must be persistent on devices in case of machine failures. If a power-off event occurs, RAIZN could traverse the metadata zone and find the correct range of PPU with the help of log headers. However, the existence of log headers introduces extra space and write bandwidth overhead, especially under workloads of small-sized requests. We evaluate the write amplification ratio (WAR) caused by log headers under 4 KB sequential write workload (using 64 KB chunk size and 3+1 stripe), finding that the WAR of the RAIZN is 2.98 since almost every PPU needs the same size log header. So the performance penalty caused by log headers cannot be ignored.

Implications. Updating parity out of place introduces log headers, consuming storage space and write bandwidth.

C. Zone Random Write Area (ZRWA)

ZRWA has been widely supported by mainstream ZNS SSDs (Table I) and been included in NVMe specification in 2024 [59]. For a zone, ZRWA is a fixed-size contiguous LBAs that can receive random, concurrent overwrite requests. The write pointer of a zone in ZRWA mode points to the start of ZRWA. Figure 5 illustrates a typical implementation of ZRWA, demonstrating its working principle. Applications or file systems perceive a set of logical zones or LBAs. Since the size of one flash page is usually greater than that of a single LBA, ZNS SSDs typically employ RAM-based write buffers to gather enough LBAs before writing data to flash pages. The write buffers permit repeated overwrites, and a zone can expose the write buffer to the host via ZRWA.

ZRWA can be committed implicitly (*i.e.*, move across LBAs) by writing data to an LBA that exceeds the area. In Figure 5, given that the size of a flash page is 4 LBAs and the size of the write buffer is 8 LBAs, a write at LBA_{11} will cause the data in the first 4 LBAs to be flushed to flash pages. After that, LBA range [0,3] can no longer be overwritten any more. It should be noted that overwrites within ZRWA do not move WP. ZNS SSD defines the size of write buffers via the field of ZRWA size and the minimal number of LBAs flushed to flash pages via the field of ZRWAFG (Table I).

A zone can be opened in ZRWA mode via explicit open operations [9]. A zone in ZRWA mode cannot switch to the traditional mode unless it is reset. We refer to zones in ZRWA mode as *ZRWA zones*, and zones in traditional mode as *non-ZRWA zones* in the rest of the paper. The data on ZRWA (or write buffers) can be durable against a machine failure if Power-Loss Protection (PLP) is supported by the SSD. Most server ZNS SSDs today support PLP [27].

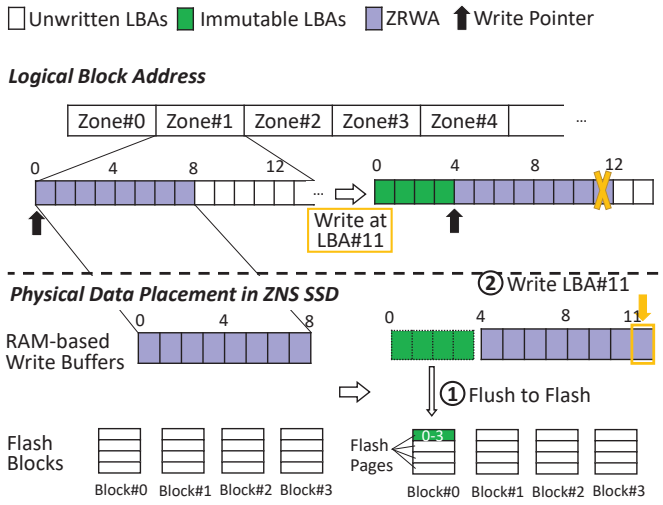


Fig. 5. ZRWA movement when a write at LBA_{11} occurs. Writes at the LBA range [0,7] (purple grids) fall in ZRWA and do not move ZRWA. But a 1 LBA-sized write at LBA_{11} falls in unwritten LBAs (white grids) and moves the tail of ZRWA to LBA_{11}. Meanwhile LBAs of range [0,3] (green grids) are flushed from RAM-based write buffers to flash pages.

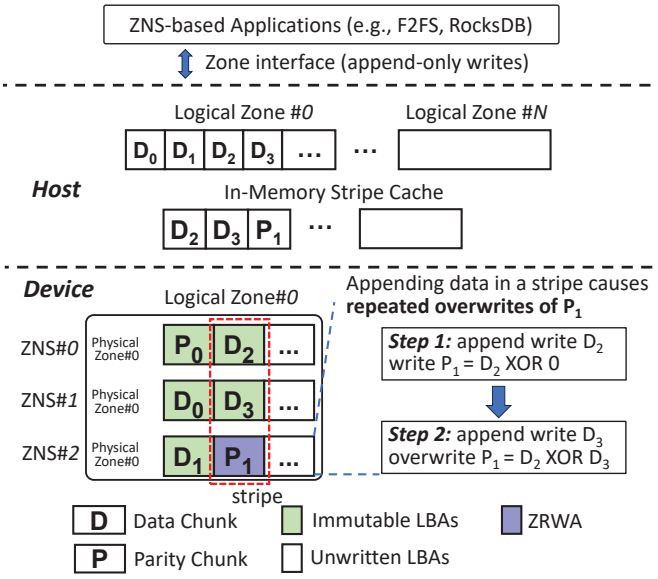


Fig. 6. Zebra architecture.

III. Zebra Design and Implementation

We propose Zebra, a general efficient redundant all-ZNS-SSD RAID enabled by ZRWA (§III-A). In this section, we first introduce the architecture of Zebra. Then we show how Zebra locates WP with lightweight metadata (§III-B), together with how Zebra recovers from disk failures (§III-C1) and machine failures (§III-C2). Finally, Zebra accelerates memory-related operations and handles flush commands (§III-D).

A. Architecture Overview and Key Idea

Figure 6 illustrates the architecture of Zebra. Zebra is a generic framework for ZNS RAID, exposing logical zone interfaces to applications. Zebra combines the physical zones from different ZNS SSDs into a logical zone, and thus data chunks in each logical zone are distributed uniformly over ZNS SSDs.

The **key idea** of Zebra is to utilize ZRWA to accommodate parity chunks, as we find the random write property of ZRWA within a certain sliding window is a natural fit for ZNS RAID's PPUs which feature repeated overwrites within a chunk. First, ZRWA permits repeated overwrites within the ZRWA window. Similarly, the parity needs repeated overwrites and its destination is always within the parity chunk before the stripe becomes full. For example in Figure 6, appending D_2 generates P_1 and appending D_3 causes P_1 to be overwritten. Second, ZRWA can move to the next window by implicitly committing. When a stripe becomes full, the old parity chunk (e.g., P_0 of Figure 6) becomes immutable and the next parity chunk moves to P_1.

Additionally, two reasons make ZRWA efficient and practical for PPUs. First, each physical zone has a private ZRWA, and therefore it can accommodate PPUs independently, avoiding a centralized metadata zone and GC. Second, the ZRWA size of existing ZNS SSDs (Table I) exceeds the typical chunk size (64 KB), ensuring that PPUs can fall into ZRWA.

Host-side data structure. Zebra maintains WPs and states of *active* logical zones in the host memory. In-memory stripe cache is also maintained at the host side, containing all stripes of *active* logical zones. This design avoids extra read requests to ZNS SSDs for PPU calculation and mitigates the request dependency of RAID writes.

Device-side data layout. All physical zones in Zebra are opened in ZRWA mode and can store parity chunks in any style (e.g., left-asymmetric). The chunks of stripes are distributed uniformly over all ZNS SSDs, indicating that data chunks and parity chunks coexist in each zone. Zebra supports different RAID setups (*e.g.*, RAID 5, RAID 6) and we take RAID 5 as an example in this paper.

Handling partial stripe append writes. Figure 7 shows the executing process of 5 write requests and the corresponding movement of ZRWA in a (3+1) Zebra. The size of the write W_2 equals the chunk size while that of the remaining 4 writes equals half of the chunk size. In the initial state, logical zone WP points to the beginning of D_3 (*i.e.*, the start of the second stripe). When processing W_0, Zebra appends data to D_3 and partial parity to the first half of P_1, moving the tail of ZRWA of both ZNS_0 and ZNS_3 to the midpoint of D_3 and P_1, respectively. Then W_1 comes, Zebra continues appending W_1 to D_3 and partial parity to P_1, moving ZRWA of ZNS_0 and ZNS_3 as well. When W_2 comes, Zebra appends W_2 data to D_4, moving ZRWA of ZNS_1 implicitly. Here Zebra overwrites the PPU triggered by W_2 to P_1 without moving ZRWA of ZNS_3 because the parity chunk P_1 has fallen in ZRWA. Lastly, when W_3 and W_4 come, Zebra writes data to D_5 and overwrites the corresponding 2 PPUs to P_1 sequentially, reaching a state that all tails of ZRWA move to the end of their chunks in the stripe. It should be noted that the I/O pattern of parity chunks (*e.g.*,

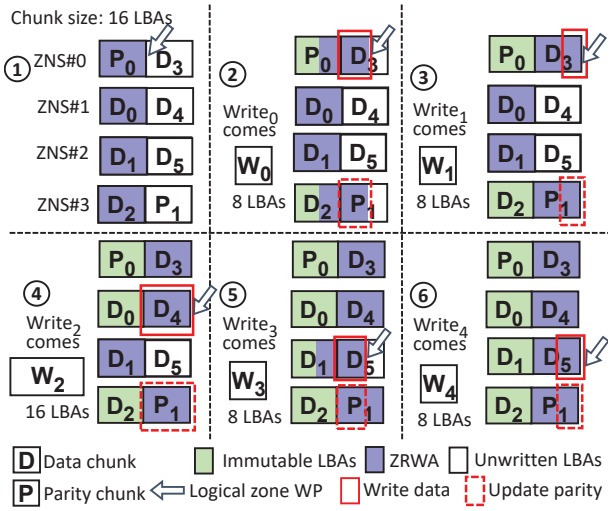


Fig. 7. The executing process of 5 write requests and the movement of ZRWA in Zebra with a (3+1) RAID5 setup. The I/O pattern of parity chunks (e.g., P_1) on ZRWA is similar to repeated sequential overwrites.

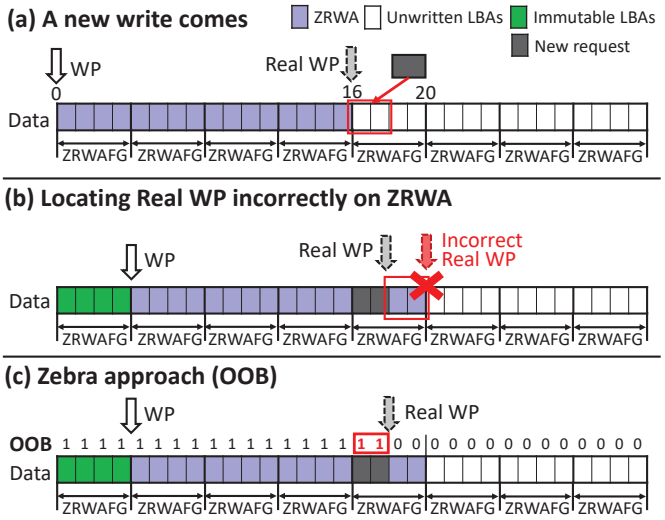


Fig. 8. Moving process of WP on ZRWA. (a) A 2 LBA-sized write request is issued to LBA_{16}. (b) The write request results in a 4-LBA movement of ZRWA and it is hard to locate the tail of the request (*i.e.*, Real WP). (c) Zebra uses lightweight metadata on OOB to locate Real WP.

P_1) on ZRWA is similar to **repeated sequential overwrites**.

Employing ZRWA in Zebra is not a free lunch. Several reliability issues related to ZRWA must be addressed. The first is the difficulty of locating WP on ZRWA against a machine failure, we design lightweight metadata for ZRWA-based WP to ensure crash consistency (§III-B). The second is how to avoid storage space reduction after a power-off event, we give the recovery process of Zebra (§III-C).

B. Locating WP with Lightweight Metadata

The different moving granularity of ZRWA zones compared to non-ZRWA zones makes it challenging to locate the Write Pointer (WP) after a machine failure. Incorrect storage and retrieval of the WP can result in data loss and issues with data durability. This section introduces lightweight metadata to locate WP, to ensure the correctness of Zebra.

Distinction of Moving Granularity. In non-ZRWA zones, newly-written data moves WP and the minimum moving unit is 1 LBA (*i.e.*, 4 KB), which equals the write granularity of ZNS SSDs. Unlike non-ZRWA zones, the ZRWA zone moves in the unit of ZRWAFG [7], which usually equals the size of a flash page in SSD (*i.e.*, over 16 KB for most MLC/TLC SSDs [8], [55], [64]). For example (Figure 8(a)), if the ZRWAFG is 4 LBAs and a 2 LBA-sized append write request is issued to LBA_{16}, the start of ZRWA moves to LBA_4 (and hence the tail of ZRWA to LBA_{20}), even though no data have been written to LBA range [18,20) in ZRWA.

Locating WP after a crash. Zebra makes ZRWA transparent to applications and provides traditional zone abstraction (*i.e.*, non-ZRWA zones) to upper applications. In non-ZRWA zones, the WP points to the tail of successfully written data. However, in ZRWA zones, the WP points to the start of ZRWA, which differs from the semantics of WP in non-ZRWA zones and is unable to locate the tail LBA of the last append write. To eliminate the semantic gap, we define **Real WP**, which points to the tail of consecutive successfully written data in ZRWA zones. Note that Real WP does not always equal the tail of ZRWA due to the coarse-grained moving units of ZRWA.

Locating Real WPs in ZRWA zones after a failure is not trivial. Considering a sudden power-off event, the cached WPs of all zones in DRAM are lost. Although WPs are maintained on ZNS devices, ZNS SSDs do not record which regions of ZRWA are written so Real WP cannot be restored accurately during recovery. If a ZNS RAID takes the tail of ZRWA as Real WP, the regions without written data (*i.e.*, LBA range [18,20) in Figure 8(b)) will be mistakenly treated as valid.

Lightweight metadata for locating Real WP. Zebra resolves the problem with out-of-band area (OOB), a proven technique [6], [52] supported by modern NVMe SSDs, which is widely adopted by industrial systems [38], [71], [78]. The OOB area is a spare area for each flash page to store the metadata of the page. The size of the OOB area often ranges from 8 B to 64 B per LBA. When writing data to a flash page, users can attach the metadata to the OOB area, and the metadata written into the OOB area is consistent with the data written to corresponding flash pages.

Zebra utilizes the metadata on the OOB area to judge if the data has been written. The OOB area is filled with 0 in the initial state. When a 2 LBA-sized write request is issued to LBA_{16} (see black grids in Figure 8(c)), the corresponding OOB area is written with 1, indicating that a write request occurs on the flash page. If a machine failure occurs at this moment and then the machine is powered on again, Zebra needs to query the WP from ZNS SSD. Recalling that ZNS SSDs maintain the WP location and zone states on hardware. By utilizing OOB, Zebra traverses the metadata of the OOB area from flash pages on ZRWA and places the Real WP at the tail of consecutive prefix pages, of which the metadata is

1 (LBA_{18} in Figure 8(c)). Thus Zebra restores the WP of the zone successfully after a power-off event.

Overhead of metadata. Storing metadata in OOB area is lightweight. Our evaluation shows that no performance degradation is observed when we write metadata to OOB areas synchronously. For space consumption, Zebra only consumes 1 bit of OOB area per LBA while the size of OOB area per LBA is always not less than 8 Bytes.

C. Fault Tolerance

Here we introduce how Zebra recovers from failures. Failures in ZNS RAID can be categorized into two types: disk failure and machine failure. A disk failure occurs when some ZNS SSDs in the RAID become inaccessible due to firmware issues or the end of their lifespan. In this case, Zebra restores the lost data on the failed device using redundant parity. The other type of failure is a machine failure, often due to a power-loss event. Assuming that data in the host-side DRAM is lost, Zebra must ensure that the data from write requests already marked as complete to applications is not lost. Additionally, Zebra must restore a prefix of successfully written but unfinished requests [58].

1) Disk-Failure Recovery: To handle a disk failure, Zebra adopts a common way to perform the recovery process. The write requests issued to the failed ZNS SSD can be skipped since the caller cannot get any response, while the read ones are performed in 'Reconstructed Read' [45], which means the read requests can be reconstructed from data and parity in the same stripe, leading to the increasing read traffic. Zebra starts the recovery process if a new ZNS SSD is ready to replace the failed one. Specifically, Zebra first reads data from all stripes and calculates the parity that resides on the failed ZNS SSD. Then Zebra writes the parity and data into the newly-joined ZNS SSD.

2) Machine-Failure Recovery: When the machine fails, data structures stored on the host memory are lost. Zebra must restore them from devices after a machine failure. Specifically, the data structures contain in-memory stripe cache, logical zone states and WPs. We only discuss the recovery of WPs of logical zones since others can be rebuilt by WPs. After a machine failure, Zebra must restore the RAID to a consistent state and maintain the append-only semantics. There are two key challenges to this recovery process.

Data inconsistency. First, it is crucial to maintain user data consistency, preventing holes or partially written data chunks in the middle of a logical zone, as Zebra concurrently sends and processes data blocks for a logical zone across multiple physical zones. For example, consider a logical zone spanning three physical zones, each 16 KB in chunk size. If an incoming 32 KB write request generates two 16 KB updates of data chunks and distributes them to two physical zones, a power-off event could result in the second data chunk being fully durable while only the first 4 KB of the first data chunk is written. This would leave a 12 KB gap in the logical zone, leading to incorrect user data.

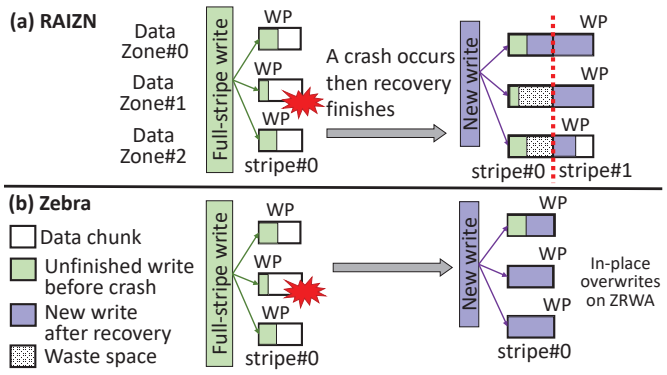


Fig. 9. Zero storage space reduction after recovery. After recovering from a failure, (a) RAIZN cannot reuse the partially-written stripe ($stripe_0$) due to dirty data on $Zone_1$ and $Zone_2$, while (b) Zebra reuses $stripe_0$ by overwriting new data on ZRWA. We omit parity in this figure for brevity.

Parity inconsistency. Second, it is important to ensure the consistency between user data and parity. User data and parity updates are concurrently processed by individual physical zones, which may lead to a mismatch between the parity value and the verification value of the corresponding data blocks.

Zebra resolves the first challenge by identifying the consistent Real WP of the data chunk and discarding any data chunks beyond that point. Once the data chunks are consistent, the parity chunk can be rebuilt from them, thus avoiding the second issue. The following outlines the detailed three steps:

- **Step 1: Querying the zone states.** Zebra issues zns-zone-management-recv command to ZNS SSDs and receives the descriptors containing the states and WPs of physical zones.
- **Step 2: Calculating the Real WPs.** The core task is to find the last and valid real WP for each logical zone. In particular, each logical zone comprises multiple physical zones. Starting from the WPs of physical zones found in step one, Zebra scans the OOB area (details in §III-B) to locate the Real WP for each physical zone. As a full-stripe append write can be divided and scattered across multiple physical zones in parallel, Zebra needs to synchronize the Real WPs of these physical zones to determine the Real WP of the logical zone. The synchronization is straightforward: starting from the first physical zone, Zebra takes the first Real WP that has not reached the end of its data chunk (i.e., the data chunk is not full).
- **Step 3: Synchronizing the data and parity.** Zebra discards invalid data with LBAs beyond the real WP of the logical zone by overwriting them with zero, as this data belongs to unfinished requests and violates the append-only semantics. Finally, Zebra recalculates the parity chunk based on the recovered data chunks.

The only exception is that both a disk failure and a machine failure occur simultaneously. A disk failure prevents Zebra from restoring the Real WPs of physical zones in the failed disk through scanning OOB area simply. To restore the Real WPs, we need to know the range of data chunks written by requests in the failed disk. Thus in Step 2, Zebra uses the OOB area of parity chunks in the unfailed disks to restore

Real WPs in the failed disk. Specifically, the OOB area of parity blocks records the latest write requests that update those blocks. By tracing back the requests, Zebra knows the region of data chunks in the failed disk that the requests update. Then Zebra restores the Real WPs of zones in the failed disk and rebuilds the RAID. To this end, Zebra extends the OOB area used for parity chunks from 1 bit to 1 Byte, which is still a moderate space consumption. Theoretically, only $\lceil \log_2 (stripe\_size/chunk\_size) \rceil$ bits are required.

Zero storage space reduction after recovery. After recovering from a machine failure, Zebra does not introduce extra storage space reduction compared to RAIZN. Figure 9 shows an example. In Figure 9, a full-stripe write request (green), *W*, is split into several sub-I/Os and issued to multiple physical zones concurrently. If a machine failure occurs before the finish of *W*, some of the sub-I/Os have been completed but others have not. In Figure 9(b), Zebra restores the logical zone WP to the WP of $Zone_0$ and cleans dirty data of unfinished writes on $Zone_1$ and $Zone_2$. After recovery, if a new write request (purple) comes, Zebra benefits from in-place updates supported by ZRWA and can directly overwrite the new request in $stripe_0$.

On the contrary, RAIZN cannot overwrite dirty chunks of $stripe_0$ on $Zone_1$ and $Zone_2$ due to the no-overwrite constraint (Figure 9(a)). To this end, RAIZN has to skip $stripe_0$ and write the new request to the next stripe (*i.e.*, $stripe_1$), leading to the permanent storage space reduction in $stripe_0$. To record the information of missing space, RAIZN has to consume extra memory overhead to maintain the mapping table.

D. Implementation Details and Discussions

Accelerating memory-related operations. Parity calculation and memory copy of stripe cache are two main memory-related operations in Zebra. With NVMe SSDs providing ultra-low I/O latency (10µs approximately) [26], the overhead of memory-related operations cannot be neglected. The latency overhead of parity calculation and memory copy in Zebra occupies 11.6% and 5.6%, respectively. To this end, Zebra adopts Advanced Vector Extensions (AVX) instructions [70], a parallel acceleration technique widely supported by modern CPUs [30], [39] to accelerate memory-related operations. Specifically, Zebra uses AVX instructions to access the data in memory and perform calculation operations via vector parallelism, which not only saturates the bandwidth of host memory but also accelerates computing. The operations of parity calculation are mainly XOR and polynomial computations, which are truly compatible with AVX instructions.

Handling Flush commands. Some applications like file systems use flush commands to ensure that previously written data blocks are durable on SSDs. For ZNS devices that support PLP, data is guaranteed to be durable once it has been transferred to the device [42]. In these SSDs, Zebra moves the real WP once a write finishes. If ZNS SSDs do not support PLP, the movement of WPs is postponed until flush completion.

Comparing to the batching approach. One popular approach to avoid PPUs is batching enough data blocks to form a full-stripe write [35], [66]. This approach has three main drawbacks. First, to ensure instant durability, batching requires other persistent devices (e.g., NVDIMM or persistent memory), which increases the operational cost and the system's heterogeneity. Second, batching increases the request latency [65], especially for requests that arrive earlier. Third, applications cannot persist multiple dependent write requests in proper order via the batching approach. For instance, to ensure crash consistency, file systems first persist the logs and then persist the log footers while journaling [42].

IV. Evaluation

A. Experiment Setup

System setup. We compare our system with RAIZN [37] through experiments. Besides RAIZN, we also implement an optimized version of RAIZN named RAIZN+, where we avoid triggering GC in RAIZN (§II-B2) and allocate one metadata zone for **each** logical zone (§II-B1). We prototype Zebra as a user-space zone-interface module of Storage Performance Development Kit (SPDK) [20] in C++ with around 5.2K LoC. For a fair comparison, the other two systems are implemented under SPDK framework as well.

Tested ZNS models. We evaluate Zebra and peer systems on both large-zone and small-zone ZNS SSDs, two typical types of ZNS models [56], [62]. A large-zone ZNS SSD distributes physical flash blocks of each zone over **all** channels and dies to fully exploit the hardware parallelism, which results in a large zone size. On the other hand, in a small-zone ZNS SSD, flash blocks of each zone are redirected to only **one or multiple** flash channels instead of all channels. Thus small-zone ZNS SSDs provide hardware isolation between zones, which is an important feature of ZNS to improve the QoS of multi-tenant performance.

First, for large-zone ZNS SSDs, we use 4 Western Digital DC ZN540 2TB ZNS SSDs to form a (3+1) RAID 5, configuring the chunk size with 64KB. Second, for small-zone ZNS SSDs, we use NVMeVirt [36] to emulate 7 small-zone ZNS SSDs and build a (6+1) RAID 5. Each emulated ZNS is configured with 8 channels with 2 dies each (default ZNS settings in NVMeVirt). We configure the small-zone ZNS SSDs with each zone spanning only one die to emulate Samsung PM1731a ZNS SSDs [61], which is also a common setting named SU-Zone in [62].

Testbed. The server we use has two 10-core CPUs and 64GB DRAM, running Ubuntu 22.04 with the kernel version of 5.15.0. The SPDK version is v22.09.

Workloads. We evaluate the performance of Zebra with micro benchmarks and application benchmarks. Micro benchmarks include write/read/read-write-mixed workloads and real-world application traces. We collect 6 popular real-world application traces [32], [74], [76]: (1) 2 traces running `YCSB` [21] on RocksDB [15], named `YCSB-A`, `YCSB-B`, (2) 1 trace running `TPC-C` [4] on MySQL [57] named `TPC-C` and (3) 3 widely-used traces from SNIA repository [5]: `Exchange` [1],

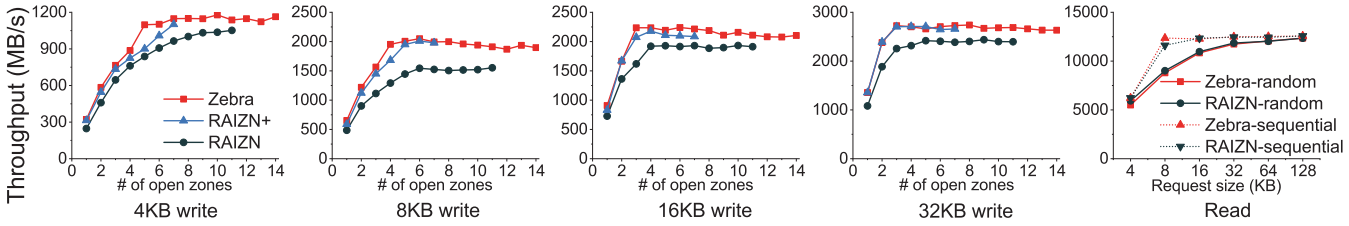


Fig. 10. Write and read throughput on large-zone ZNS SSDs with varying I/O sizes and the number of open zones.

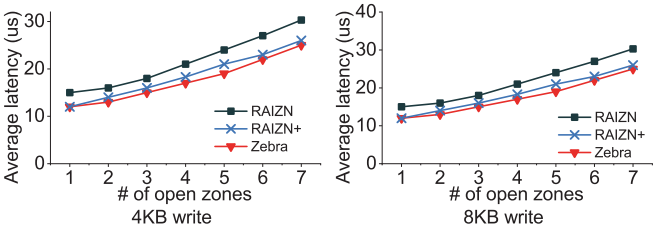


Fig. 11. Average write latency on large-zone ZNS SSDs.

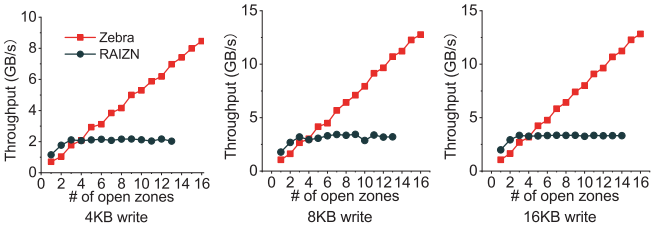


Fig. 12. Write throughput on small-zone ZNS SSDs.

`Proxy` [2], [3] and `VirtualDesktop` [41]. For application benchmarks, we build a key-value store, RocksDB atop of ZNS RAID and evaluate its performance via native tool `db_bench`. We list the detailed workload patterns in Table II.

B. Micro Benchmarks

Write workloads on large-zone ZNS. We evaluate the throughput and average/p99 latency under write workloads, by varying request size and the number of open zones. Figure 10 shows the throughput with varying the number of open logical zones on large-zone ZNS. The maximum number of logical zones for the device is 14. RAIZN can only use 11 for data zones as RAIZN has to reserve 3 metadata zones [37]. RAIZN+ can only open 7 logical zones concurrently for evaluation because each open logical zone demands a metadata zone.

Under 4 KB write workload (Figure 10), Zebra increases the throughput by 30% compared to RAIZN when using 1 open zone. When the open zone count rises, Zebra outperforms RAIZN by 8%-31% in throughput because Zebra can update parity in place and does not require GC as in RAIZN. Moreover, Zebra requires 5 open zones to reach the peak throughput (1178 MB/s) under 4 KB workload while RAIZN requires 9 open zones to reach the maximum. This indicates that with the same amount of open zones, applications built on Zebra achieve higher throughput than on RAIZN. Zebra improves the throughput by 3%-22% compared to RAIZN+ slightly due to the optimization of memory-related operations. Note that RAIZN+ only serves as a performance reference. It is unrealistic and cannot support long-time running since it generates obsolete PPUs but never performs GC.

When the request size rises, Zebra outperforms RAIZN by 34%/17%/14% on average under 8KB/16KB/32KB write workload, respectively. Compared to RAIZN, Zebra eliminates metadata zones and avoids GC so it reduces the write amplification of ZNS RAID and boosts the performance. When the request size reaches 192 KB, PPUs disappear in ZNS RAID because all requests are performed in full-stripe writes, so the three systems reach the physical upper limit of ZNS SSDs (around 3400 MB/s). Zebra further cuts the average latency by 19%/20% compared to RAIZN under 4 KB and 8 KB workloads, respectively (Figure 11). The contention of multi-zone PPU aggregation causes the metadata zones in RAIZN to become a bottleneck, resulting in increased average latency.

Write workloads on small-zone ZNS. We also evaluate the write throughput of Zebra and RAIZN on small-zone ZNS SSDs (Figure 12). On small-zone ZNS SSDs, the bandwidth of a zone is strictly limited to the bandwidth of the channel it is located on. When the open zone count is smaller than 4, RAIZN outperforms Zebra in throughput slightly. It is because RAIZN uses more zones (both data and metadata zones) than Zebra (only data zones). When the open zone count increases, thanks to all data zones receiving both write requests and PPUs, the throughput of Zebra grows linearly while that of RAIZN is limited by metadata zones. Therefore, Zebra improves the write throughput by 4.2×/3.9×/3.3× at most under 4 KB / 8 KB / 16 KB write workloads, respectively.

The experiments on small-zone ZNS SSDs can better demonstrate the advantage of 'all-to-all' architecture in Zebra. In a large-zone ZNS SSD, zones share the bandwidth of all

TABLE II
I/O CHARACTERISTICS OF EVALUATED WORKLOADS

Workloads	Write ratio	Avg. request size (KB)	Dataset range (GB)
`Read`	0%	4 / 8 / 16 / 32	200
`Write`	100%	4 / 8 / 16 / 32	40
`Mixed`	0%-100%	4 / 8 / 16	200
`YCSB-A`	64%	189.2	478.5
`YCSB-B`	62%	197.1	419.2
`TPC-C`	75%	30.4	444.0
`VirtualDesktop`	42%	25.5	1734.4
`Exchange`	70%	14.1	478.5
`Proxy`	32%	12.9	65.3

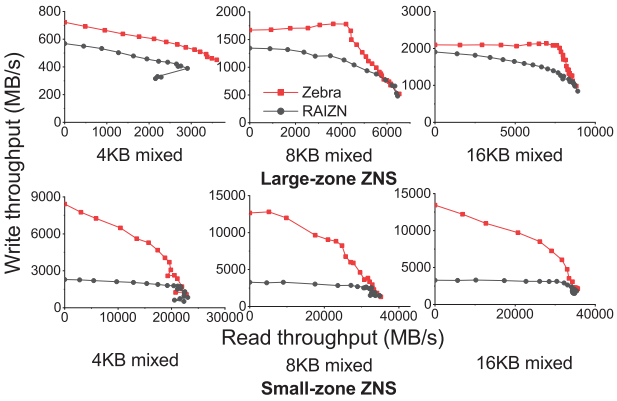


Fig. 13. Throughput under read-write-mixed workloads with varying read/write ratio (top three - large-zone ZNS, bottom three - small-zone ZNS).

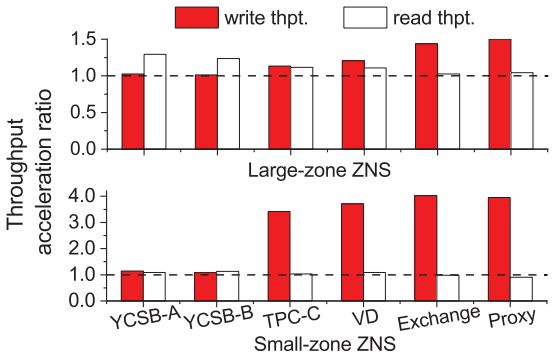


Fig. 14. Read/write throughput acceleration ratio (normalized to RAIZN) under real-world I/O traces on large-zone and small-zone ZNS SSDs.

channels and there is no hardware isolation between zones. So metadata zones of RAIZN in large-zone ZNS can leverage all channels to store PPU, which reduces the impact of the contention of multi-zone PPU aggregation. However, on small-zone ZNS SSDs, RAIZN, which uses an 'all-to-one' architecture, is heavily constrained by the bandwidth of metadata zones and shows poor performance.

Read workloads. Figure 10 shows that Zebra achieves comparable performance to RAIZN under both sequential and random read workloads since Zebra mainly optimizes the write performance. The sequential and random read throughput is similar in Zebra under 4 KB workload, but the sequential one outperforms the random one by up to 40% when the request size increases. When the request size is larger than 64 KB, the sequential and random throughput converge again.

Mixed workloads. We evaluate the performance of Zebra under workloads with a mixed ratio of read and write requests. We generate workloads with 4KB, 8KB and 16KB request size, but for each workload, the request size remains unchanged. We change the read request ratio ranging from 0 to 0.9 and plot the changing curve of read/write throughput in Figure 13. On large-zone ZNS, the performance curve of Zebra always wraps around that of RAIZN from the top right. It means that Zebra achieves a higher write throughput than RAIZN when the read throughput of the two systems are equivalent. On small-zone ZNS, the performance advantage of Zebra becomes more obvious. When write requests dominate mixed workloads, Zebra outperforms RAIZN in write throughput significantly, by 2.2× on average. The experiment shows that Zebra still achieves write throughput improvement when workloads are mixed with read requests.

Real-world application traces. We collect 6 real-world application traces and replay these traces on Zebra and RAIZN. Figure 14 shows the write/read throughput acceleration ratio under application traces. The acceleration ratio is calculated by dividing the throughput of Zebra by that of RAIZN. For workloads with large-size requests (`YCSB-A` and `YCSB-B`), Zebra achieves similar write throughput with RAIZN because write requests are usually processed as full-stripe writes. The average request size of the remaining 4 workloads does not exceed 32 KB. Under workloads with small-sized requests, Zebra improves the write throughput by 1.3× and 3.8× on average on large-zone and small-zone ZNS, respectively, due to resolving the PPU issue.

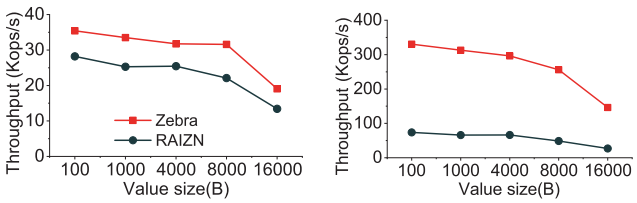
C. Application Benchmarks

We adapt the storage engine ZenFS to SPDK with moderate modifications and build RocksDB atop of ZNS RAID with modified ZenFS. We run 5 workloads using `db_bench` (*i.e.*, *fillsync*, *fillseq*, *fillrandom*, *overwrite*, and *readwhilewriting*). The *fillsync* workload performs `INSERT` operations in sync mode, which ensures that the log of `INSERT` operation persists on the Write Ahead Log (WAL) file via `fsync()` before returning to users. Besides, the other 4 workloads run in async mode, indicating that the WAL file is persisted on ZNS RAID in a batch of `INSERT` operations. The evaluated value size ranges from 100 B to 16000 B.

Under *fillsync* workload (Figure 15), Zebra outperforms RAIZN in RocksDB throughput by 24%-42% on large-zone ZNS and by 4.5×-5.4× on small-zone ZNS. It is because RocksDB issues many small-sized write requests to ZNS RAID in sync mode. When the value size is smaller than 4000 B, the throughput is similar because RocksDB pads the log into 4KB alignment in the WAL file. Meanwhile, Zebra brings an up to 10% reduction of average latency compared to RAIZN, especially when the value size exceeds 8000 B. When evaluating under other 4 workloads, Zebra achieves the operation latency close to RAIZN because `INSERT` operations return immediately after writing to Memtable in DRAM. However, benefiting from the high throughput of Zebra, RocksDB based on Zebra improves the throughput by 17.7% on average under the 4 workloads compared to RAIZN.

D. Impact of Individual Techniques

We show the performance improvement brought by individual key techniques in Zebra. First, introducing ZRWA in ZNS RAID brings the most performance gain. Figure 12 is such an example, indicating that Zebra increases the throughput by 4.2× at most compared to RAIZN.



(a) Large-zone ZNS SSD (b) Small-zone ZNS SSD

Fig. 15. RocksDB throughput under *fillsync* workload.

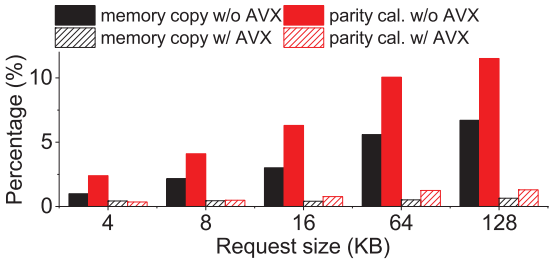
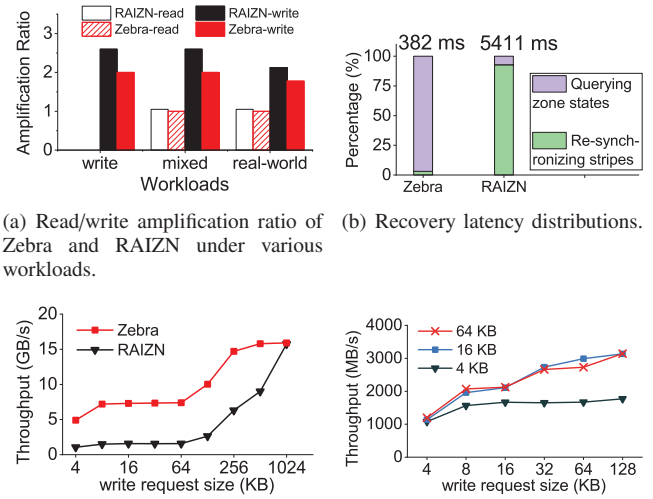


Fig. 16. Effects of the AVX optimization on memory-related operations.



(a) Read/write amplification ratio of Zebra and RAIZN under various workloads.

(b) Recovery latency distributions.

(c) Throughput of RAIZN and Zebra with the configuration of (5+2) RAID-6 on small-zone ZNS SSDs.

(d) Throughput of Zebra under write workloads with 3 settings of the chunk size (4 KB / 16 KB / 64 KB) on large-zone ZNS SSDs.

Fig. 17. Experiment results.

Second, to show the benefits of AVX acceleration, we implement a baseline version of Zebra and add the technique into Zebra separately. Figure 16 shows the latency percentage of memory copy and parity calculation operations in the entire write process of Zebra. With AVX optimization on, the latency percentage of two operations decreases by 6.1% and 10%, respectively.

Last, we show the average read/write amplification ratios of Zebra and RAIZN under different types of workloads in Figure 17(a). Zebra reduces the write amplification ratio by 19%-30% under various workloads as Zebra eliminates RAID-level GC and PPU log headers compared to RAIZN. The remaining performance improvement of Zebra results from avoiding the bottleneck of metadata zones.

E. Recovery Performance

Disk-failure recovery. The disk-failure recovery time to rebuild a ZNS SSD in Zebra offline is linearly correlated to the amount of data to repair, reaching 899 seconds with 1000 GB of data to repair. Besides, we evaluate the degraded read performance of Zebra when a ZNS SSD is failed. We further evaluate the read throughput of degraded Zebra with a failed device, reaching 6.1 GB/s and 9.2 GB/s under 64 KB / 128 KB workloads, respectively.

Machine-failure recovery. Figure 17(b) shows that Zebra brings a 14.2× reduction of machine-failure recovery time compared to RAIZN. Our zoom-in experiment shows that the time to query the states of ZNS SSDs (Step 1 in §III-C2) is the same. RAIZN spends lots of time (93% of the total recovery time) traversing metadata zones to search for valid PPUs. On the contrary, Zebra re-synchronizes the partially-written stripes via logical zone WP, avoiding the step of reading the entire metadata zones.

F. System Overheads and Sensitivity Analysis

In-Memory Space Overhead. In Zebra, in-memory stripe cache only maintains the stripes that WPs of *active* logical zones point to. Each active logical zone only has one stripe cached at host side. Thus the memory space consumption of the stripe cache is $maximum\_active\_zone\_count \times stripe\_size$ (3.5MB in our experiments).

Performance of RAID 6. With the configuration of (5+2) RAID 6, we evaluate the throughput of Zebra and RAIZN on small-zone ZNS SSDs in Figure 17(c). Under write workloads with the request size smaller than 64 KB, Zebra outperforms RAIZN in throughput by 4.7× on average. With the increasing write request size, the throughput of RAIZN and Zebra converge slightly because large-sized write requests hardly generate PPUs.

Impcat of chunk size. Figure 17(d) shows the write throughput of Zebra with different request sizes (ranging from 4 KB to 128 KB) based on large-zone ZNS SSDs when setting the chunk size to 4 KB / 16 KB / 64 KB. With the 4 KB chunk size, Zebra shows a 29% throughput decrease on average compared to the 64 KB one because SSDs are IOPS-limited under workloads with small-sized requests and cannot achieve the peak bandwidth. When using the 16 KB chunk size, Zebra improves the throughput slightly compared to the 64 KB one. However, the improvement comes at the cost of a 58% increase of latency because Zebra has to split a request to many sub-I/Os of which the size equals the chunk size. Thus 64 KB is an optimal configuration for the chunk size.

G. Comparison with Other Systems

Comparison against RAIZN with more metadata zones. One might question whether increasing the count of metadata zones is a straightforward and effective solution to address the PPU problem in RAIZN. We show that adding the metadata

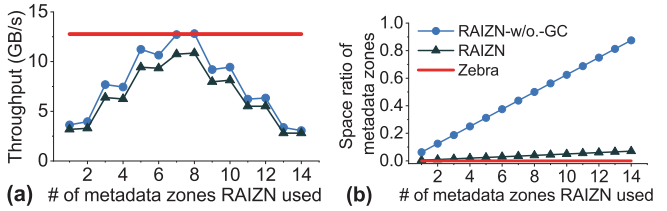


Fig. 18. (a) Throughput and (b) the ratio of metadata zone space overhead to the total volume in Zebra, RAIZN-w/o.-GC and RAIZN with the varying metadata zone count.

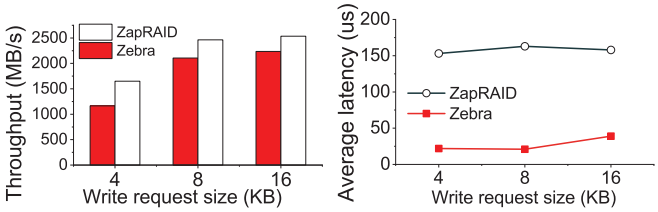


Fig. 19. Performance comparison with ZapRAID under write workloads

zone count in RAIZN incurs extra space overhead or harms the throughput compared to Zebra. Here we implement a version of RAIZN named RAIZN-w/o-GC for comparison, which does not reclaim obsolete PPUs in RAIZN. Figure 18(a) shows the upper limits of RAIZN and RAIZN-w/o-GC throughput with the varying metadata zone counts under 8 KB write workloads on small-zone ZNS SSDs. As a reference, we also plot the throughput of Zebra with the same total amount of zones. When using 8 metadata zones, RAIZN reaches its maximum throughput, but this is still 14% less compared to Zebra. Note that more metadata zones come at the cost of fewer active data zones and reduce the running application count that ZNS RAID systems can support concurrently. RAIZN-w/o-GC achieves the similar throughput with Zebra because it does not trigger GC in RAIZN. However, the performance improvement comes at the cost of huge space overhead. Figure 18(b) shows the space overhead of metadata zones to the total volume in the three systems. Compared to RAIZN and Zebra, the space overhead of metadata zones in RAIZN-w/o-GC grows linearly since it does not reclaim obsolete PPUs in metadata zones.

Comparison against ZapRAID. ZapRAID [66] is a ZNS RAID system using zone-append interface [11] to achieve high write parallelism. Although ZapRAID and Zebra both target ZNS RAID, they are designed in fundamentally different ways. First, ZapRAID provides a block interface to applications, whereas Zebra provides a ZNS interface, making fair comparisons challenging. Second, Zebra targets the PPU issue, while ZapRAID sidesteps the issue by aggregating small-sized requests into full-stripe writes [46]. Nonetheless, we strive to compare Zebra with ZapRAID as thoroughly as possible for the sake of comprehensive evaluation.

ZapRAID outperforms Zebra in throughput by 24% on average under write workload (Figure 19). ZapRAID trades latency for high throughput by batching requests and issuing only full-stripe writes to ZNS RAID, thus avoiding PPUs altogether. In contrast, PPUs in Zebra are directly sent the the RAID without batching, and this results in lower throughput compared to larger requests. However, Zebra achieves a 4.1×-7.8× latency reduction compared to ZapRAID. This is because, when ZapRAID aggregates requests into a stripe, some requests experience queuing delays.

V. Related Work

ZNS-based storage systems. With the emergency of NVMe ZNS specification [58] and real ZNS SSDs [12], new storage system designs for ZNS SSDs roughly fall into two groups: (1) optimizing host-level GC by redesigning the data placement strategy or zone cleaning strategy , and (2) improving (write) performance with new features of ZNS . Some studies redesign the data placement strategy [33], [43] or zone cleaning strategy [16] to accelerate host-level GC. ZNS+ [24] exploits in-storage zone compaction to optimize GC. Researchers in the second group leverage the intra-zone or inter-zone parallelism to improve performance. Some work [23], [44], [66] exploit intra-zone parallelism to optimize the filesystem and RocksDB. eZNS [56] proposes an elastic-zoned abstraction adaptable to workloads. ZMS [27] adapts ZNS to mobile environments and provides limited support for overwrites in ZNS-based file systems. BIZA [72] utilizes ZRWA to cache both data and parity updates in RAID. Compared to BIZA, Zebra resolves the problem of crash consistency when introducing ZRWA in ZNS RAID.

SSD RAID. SSD RAID has been extensively studied. Some studies follow block-interface RAID [25], [45], [67], [73] while others studies [17], [31], [35] propose Log-RAID to improve write performance. Log-RAID conducts full-stripe writes on SSD RAID in an append-only manner since random writes incur device-level GC. However, Log-RAID encounters the PPU issue and three approaches are proposed: (1) buffering PPUs on persistent memory (PM) [18], [19], [29], (2) organizing PPUs into elastic-width stripes [14], [34], [53], and (3) creating extra zones receiving PPUs [37]. Compared with prior work, Zebra avoids both the involvement of PM to all-SSD RAID and the increased stripe management complexity.

VI. Conclusion

We propose, implement and evaluate Zebra, a ZRWA-enabled redundant array of ZNS SSDs achieving fault tolerance, high efficiency and ZNS abstraction simultaneously. We leverage ZRWA, a new feature of ZNS that supports random writes, and adopt it in ZNS RAID systems to resolve the PPU issue. Additionally, we address the challenges introduced by ZRWA through lightweight metadata management and fast recovery mechanism. Evaluations show that Zebra significantly outperforms existing ZNS RAID systems.

Acknowledgment

We thank all reviewers for their valuable comments. We also thank Qiuping Wang for useful discussions. This work is partially supported by National Natural Science Foundation of China under Grants 62025203 and 62302257.

References

[1] "Microsoft Enterprise Traces," http://iotta.snia.org/traces/130, 2007.

[2] "Microsoft Production Server Traces," http://iotta.snia.org/traces/158, 2007.

[3] "MSR Cambridge Traces," http://iotta.snia.org/traces/388, 2007.

[4] "TPC Benchmark C." http://www.tpc.org/tpcc/, 2010.

[5] "SNIA Block I/O Traces," http://iotta.snia.org/tracetypes/3, 2017.

[6] N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, and R. Panigrahy, "Design tradeoffs for SSD performance," in *2008 USENIX Annual Technical Conference (USENIX ATC 08)*, 2008.

[7] M. Allison, "What's New in NVMe Technology: Ratified Technical Proposals to Enable the Future of Storage," https://nvmexpress.org/wp-content/uploads/Whats-New-in-NVMe-Technology-Ratified--Proposals-to-Enable-the-\Future-of-Storage-1.pdf, 2023.

[8] A. I. Alsalibi, S. Mittal, M. A. Al-Betar, and P. B. Sumari, "A survey of techniques for architecting SLC/MLC/TLC hybrid Flash memory–based SSDs," *Concurrency and Computation: Practice and Experience*, vol. 30, no. 13, p. e4420, 2018.

[9] archlinux, " NVME-ZNS-OPEN-ZONE," https://man.archlinux.org/man/extra/nvme-cli/nvme-zns-open-zone.1.en, 2023.

[10] H. Bae, J. Kim, M. Kwon, and M. Jung, "What you can't forget: exploiting parallelism for zoned namespaces," in *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, 2022, pp. 79–85.

[11] M. Bjørling, "Zone append: A new way of writing to zoned storage," *Santa Clara, CA, February. USENIX Association.*, 2020.

[12] M. Bjørling, A. Aghayev, H. Holmberg, A. Ramesh, D. Le Moal, G. R. Ganger, and G. Amvrosiadis, "ZNS: Avoiding the block interface tax for flash-based SSDs," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 689–703.

[13] M. Bjørling, "Zoned namespaces (ZNS) SSDs: Disrupting the storage industry," https://snia.org/sites/default/files/SDC/2020/075-Bj%C3%B8rling-Zoned-Namespaces-ZNS-SSDs.pdf, 2020.

[14] J. Bonwick and B. Moore, "ZFS: The Last Word in File Systems," https://www.snia.org/sites/default/orig/sdc_archives/2008_presentations/monday/JeffBonwick-BillMoore_ZFS.pdf, 2008.

[15] D. Borthakur, "RocksDB: A persistent key-value store," https://rocksdb.org/, 2014.

[16] S. Byeon, J. Ro, S. Jamil, J.-U. Kang, and Y. Kim, "A free-space adaptive runtime zone-reset algorithm for enhanced zns efficiency," 2023.

[17] T.-c. Chiueh, W. Tsao, H.-C. Sun, T.-F. Chien, A.-N. Chang, and C.-D. Chen, "Software orchestrated flash array," in *Proceedings of International Conference on Systems and Storage (SYSTOR'14)*. ACM, 2014, pp. 1–11.

[18] C.-C. Chung and H.-H. Hsu, "Partial parity cache and data cache management method to improve the performance of an SSD-based RAID," *IEEE Transactions on Very Large Scale Integration (VLSI'14) Systems*, vol. 22, no. 7, pp. 1470–1480, 2014.

[19] J. Colgrove, J. D. Davis, J. Hayes, E. L. Miller, C. Sandvig, R. Sears, A. Tamches, N. Vachharajani, and F. Wang, "Purity: Building fast, highly-available enterprise flash storage from commodity components," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD'15)*. ACM, 2015, pp. 1683–1694.

[20] S. community, "Storage Performance Development Kit," https://spdk.io/, 2023.

[21] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM symposium on Cloud computing (SoCC'10)*. ACM, 2010, pp. 143–154.

[22] DapuStor, " DapuStor J5100," https://en.dapustor.com/product/10.html, 2023.

[23] J. Y. Ha and H. Y. Yeom, "zCeph: Achieving High Performance On Storage System Using Small Zoned ZNS SSD," in *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 2023, pp. 1342–1351.

[24] K. Han, H. Gwak, D. Shin, and J. Hwang, "ZNS+: Advanced zoned namespace interface for supporting in-storage zone compaction," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 147–162.

[25] M. Hao, G. Soundararajan, D. Kenchammana-Hosekote, A. A. Chien, and H. S. Gunawi, "The Tail at Store: A Revelation from Millions of Hours of Disk and SSD Deployments," in *14th USENIX Conference on File and Storage Technologies (FAST 16)*, 2016, pp. 263–276.

[26] B. Harris and N. Altiparmak, "Ultra-Low Latency SSDs' Impact on Overall Energy Efficiency," in *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*, 2020.

[27] J.-Y. Hwang, S. Kim, D. Park, Y.-G. Song, J. Han, S. Choi, S. Cho, and Y. Won, "ZMS: Zone Abstraction for Mobile Flash Storage," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 173–189.

[28] S. Im and D. Shin, "Flash-aware raid techniques for dependable and high-performance flash memory ssd," *IEEE Transactions on Computers*, vol. 60, no. 1, pp. 80–92, 2010.

[29] S. Im and D. Shin, "Flash-Aware RAID Techniques for Dependable and High-Performance Flash Memory SSD," *IEEE Transactions on Computers*, vol. 60, pp. 80–92, 01 2011.

[30] Intel, "Intel Intrinsics Guide," https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html, 2023.

[31] N. Ioannou, K. Kourtis, and I. Koltsidas, "Elevating commodity storage with the salsa host translation layer," in *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2018, pp. 277–292.

[32] T. Jiang, G. Zhang, Z. Huang, X. Ma, J. Wei, Z. Li, and W. Zheng, "Fusionraid: Achieving consistent low latency for commodity ssd arrays," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 355–370.

[33] J. Jung and D. Shin, "Lifetime-leveling lsm-tree compaction for zns ssd," in *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, 2022, pp. 100–105.

[34] J. Kim, J. Lee, J. Choi, D. Lee, and S. H. Noh, "Improving SSD reliability with RAID via elastic striping and anywhere parity," in *2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'13)*. IEEE, 2013, pp. 1–12.

[35] J. Kim, K. Lim, Y. Jung, S. Lee, C. Min, and S. H. Noh, "Alleviating garbage collection interference through spatial sseparation in all flash arrays," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 799–812.

[36] S.-H. Kim, J. Shim, E. Lee, S. Jeong, I. Kang, and J.-S. Kim, "NVMeVirt: A Versatile Software-defined Virtual NVMe Device," in *21st USENIX Conference on File and Storage Technologies (FAST 23)*, 2023, pp. 379–394.

[37] T. Kim, J. Jeon, N. Arora, H. Li, M. Kaminsky, D. G. Andersen, G. R. Ganger, G. Amvrosiadis, and M. Bjørling, "RAIZN: Redundant Array of Independent Zoned Namespaces," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 660–673.

[38] Lanner, "Exploring the Power of Out-of-Band (OOB) Management," https://www.lannerinc.com/news-and-events/eagle-lanner-tech-blog/exploring-the-power-of-out-of-band-oob-management, 2023.

[39] M. Larabel, "AMD Zen 4 AVX-512 Performance Analysis On The Ryzen 9 7950X," https://www.phoronix.com/review/amd-zen4-avx512, 2022.

[40] C. Lee, D. Sim, J. Hwang, and S. Cho, "F2FS: A new file system for flash storage," in *13th USENIX Conference on File and Storage Technologies (FAST 15)*, 2015, pp. 273–286.

[41] C. Lee, T. Kumano, T. Matsuki, H. Endo, N. Fukumoto, and M. Sugawara, "Understanding Storage Traffic Characteristics on Enterprise Virtual Desktop Infrastructure," in *Proceedings of the 10th ACM International Systems and Storage Conference*, 2017, pp. 1–11.

[42] E. Lee, I. Son, and J.-S. Kim, "An Efficient Order-Preserving Recovery for F2FS with ZNS SSD," in *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*, 2023, pp. 116–122.

[43] H.-R. Lee, C.-G. Lee, S. Lee, and Y. Kim, "Compaction-aware zone allocation for lsm based key-value store on zns ssds," in *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, 2022, pp. 93–99.

[44] J. Lee, D. Kim, and J. W. Lee, "WALTZ: Leveraging Zone Append to Tighten the Tail Latency of LSM Tree on ZNS SSD," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 2884–2896, 2023.

[45] H. Li, M. L. Putra, R. Shi, X. Lin, G. R. Ganger, and H. S. Gunawi, "lODA: A Host/Device Co-Design for Strong Predictability Contract on Modern Flash Storage," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 263–279.

[46] J. Li, Q. Wang, S. Han, and P. P. Lee, "The Design and Implementation of a High-Performance Log-Structured RAID System for ZNS SSDs," *arXiv preprint arXiv:2402.17963*, 2024.

[47] J. Li, Q. Wang, P. P. Lee, and C. Shi, "An in-depth analysis of cloud block storage workloads in large-scale production," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2020, pp. 37–47.

[48] J. Li, Q. Wang, P. P. Lee, and C. Shi, "An in-depth comparative analysis of cloud block storage workloads: Findings and implications," *ACM Transactions on Storage*, vol. 19, no. 2, pp. 1–32, 2023.

[49] N. Li, M. Hao, H. Li, X. Lin, T. Emami, and H. S. Gunawi, "Fantastic ssd internals and how to learn and use them," in *Proceedings of the 15th ACM International Conference on Systems and Storage*, 2022, pp. 72–84.

[50] Q. Li, Q. Xiang, Y. Wang, H. Song, R. Wen, W. Yao, Y. Dong, S. Zhao, S. Huang, Z. Zhu *et al.*, "More than capacity: Performance-oriented evolution of pangu in alibaba," in *21st USENIX Conference on File and Storage Technologies (FAST 23)*, 2023, pp. 331–346.

[51] X. Liao, Y. Lu, E. Xu, and J. Shu, "Max: A Multicore-Accelerated File System for Flash Storage," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 877–891.

[52] Y. Lu, J. Shu, and W. Zheng, "Extending the lifetime of flash-based storage through reducing write amplification from file systems," in *11th USENIX Conference on File and Storage Technologies (FAST 13)*, 2013, pp. 257–270.

[53] D. Luo, T. Yao, X. Qu, J. Wan, and C. Xie, "Dvs: Dynamic variable-width striping raid for shingled write disks," in *2016 IEEE International Conference on Networking, Architecture and Storage (NAS)*. IEEE, 2016, pp. 1–10.

[54] Y. Lv, P. Jin, X. Wang, R. Liu, L. Fang, Y. Lin, and K. Guo, "Zonedstore: A concurrent zns-aware cache system for cloud data storage," in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022, pp. 1322–1325.

[55] C. Mellor, "Western Digital SSDs get into the zone," https://blocksandfiles.com/2020/11/09/western-digital-zn540-zoned-ssd/, 2020.

[56] J. Min, C. Zhao, M. Liu, and A. Krishnamurthy, "eZNS: An Elastic Zoned Namespace for Commodity ZNS SSDs," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 461–477.

[57] MySQL, https://www.mysql.com, 2022.

[58] NVMExpress, "NVMe Zoned Namespaces (ZNS) Command Set Specification," https://nvmexpress.org/wp-content/uploads/NVM-Express-Zoned-Namespace-Command-Set-Specification-1.1c-2022.10.03-Ratified.pdf, 2022.

[59] NVMExpress, "NVMe Zoned Namespaces (ZNS) Command Set Specification," https://nvmexpress.org/wp-content/uploads/NVM-Express-Zoned-Namespace-Command-Set-Specification-\Revision-1.2-2024.08.05-Ratified.pdf, 2024.

[60] S. Pan, T. Stavrinos, Y. Zhang, A. Sikaria, P. Zakharov, A. Sharma, S. S. P, M. Shuey, R. Wareing, M. Gangapuram, G. Cao, C. Preseau, P. Singh, K. Patiejunas, J. Tipton, E. Katz-Bassett, and W. Lloyd, "Facebook's tectonic filesystem: Efficiency from exascale," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, Feb. 2021, pp. 217–231. [Online]. Available: https://www.usenix.org/conference/fast21/presentation/pan

[61] Samsung, "Samsung Introduces Its First ZNS SSD With Maximized User Capacity and Enhanced Lifespan," https://news.samsung.com/global/samsung-introduces-its-first-zns-ssd-with-maximized-user-capacity-and-\enhanced-lifespan, 2021.

[62] I. Song, M. Oh, B. S. J. Kim, S. Yoo, J. Lee, and J. Choi, "Confzns: A novel emulator for exploring design space of zns ssds," in *Proceedings of the 16th ACM International Conference on Systems and Storage*, 2023, pp. 71–82.

[63] Z. Storage, "Zoned Storage Community," https://zonedstorage.io/, 2023.

[64] B. Tallis, "Micron 3D NAND Status Update," https://www.anandtech.com/show/10028/micron-3d-nand-status-update, 2016.

[65] Q. Wang, Y. Lu, J. Wang, and J. Shu, "Replicating Persistent Memory Key-Value Stores with Efficient RDMA Abstraction," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 441–459.

[66] Q. Wang and P. P. Lee, "ZapRAID: Toward High-Performance RAID for ZNS SSDs via Zone Append," in *Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2023, pp. 24–29.

[67] S. Wang, Q. Cao, Z. Lu, H. Jiang, J. Yao, and Y. Dong, "StRAID: Stripe-threaded Architecture for Parity-based RAIDs with Ultra-fast SSDs," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 915–932.

[68] WesternDigital, "dm-zap," https://github.com/westerndigitalcorporation/dm-zap, 2023.

[69] WesternDigital, "Western Digital DC ZN540," https://hypertecsp.com/wp-content/uploads/data-sheet-ultrastar-dc-zn540-1.pdf, 2023.

[70] Wikipedia, "Advanced Vector Extensions," https://en.wikipedia.org/wiki/Advanced_Vector_Extensions, 2023.

[71] G. Xie, G. Xu, G. Wang, X. Liu, R. Cao, and Y. Gao, "hubi: An optimized hybrid mapping scheme for nand flash-based ssds," in *2011IEEE 10th International Conference on Trust, Security and Privacy in Computing and Communications*. IEEE, 2011, pp. 1015–1022.

[72] S. Yi, S. Sun, L. Peng, Y. Sun, M.-C. Yang, Z. Cao, Q. Li, M. Jung, K. Zhou, and J. Zhang, "BIZA: Design of Self-Governing Block-Interface ZNS AFA for Endurance and Performance," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 313–329.

[73] S. Yi, Y. Yang, Y. Tang, Z. Zhou, J. Li, C. Yue, M. Jung, and J. Zhang, "Scalaraid: Optimizing linux software raid system for next-generation storage," in *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, 2022, pp. 119–125.

[74] S. Zeng, X. Liao, H. Guo, and Y. Lu, "Volley: Accelerating Write-Read Orders in Disaggregated Storage," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 657–673.

[75] F. Zhang, J. Huang, and C. Xie, "Two efficient partial-updating schemes for erasure-coded storage clusters," in *2012 IEEE Seventh International Conference on Networking, Architecture, and Storage*. IEEE, 2012, pp. 21–30.

[76] G. Zhang, Z. Huang, X. Ma, S. Yang, Z. Wang, and W. Zheng, "RAID+: Deterministic and Balanced Data Distribution for Large Disk Enclosures," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 279–294.

[77] J. Zhang, H. Hu, J. Chen, and Y. Zhan, "Schinfs: A file system integrating functions of the block i/o scheduler for zns ssds," in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. IEEE, 2024, pp. 324–331.

[78] X. Zhang, F. Zhu, S. Li, K. Wang, W. Xu, and D. Xu, "Optimizing performance for open-channel ssds in cloud storage system," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 902–911.

[79] Y. Zhang, P. Huang, K. Zhou, H. Wang, J. Hu, Y. Ji, and B. Cheng, "OSCA: An Online-Model Based Cache Allocation Scheme in Cloud Block Storage Systems," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 785–798.

[80] Y. Zhou, E. Xu, L. Zhang, K. Karkra, M. Barczak, W. Gao, W. Malikowski, M. Kozlowski, Ł. Łasek, R. Lu *et al.*, "Csal: the next-gen local disks for the cloud," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 608–623.